

大小写: **None, not, class**
Library: need to be import
"Import sys" command line argument
 sys.argv: the list contains the command line argument,
 index 0 对应的一般是 xxx.py
doctest 看后面

Built-in Function:
input(): get terminal user's input as a string.
 x = input("give me x")
 type(x) == str
len(): find the length. len([1,2]) == 2; len("1") = 1
isinstance(object, class): str, int, float, bool, list, dict, tuple.
range(start, end, step): start is inclusive, end is exclusive.
Terminal Command: pwd (Print the current working directory);
ls(List the contents of the current directory); **ls -a**(Additionally show hidden files); **mkdir** comp9001 (Make a new directory named Comp9001); **cd** comp9001 (Change current directory to Comp9001); **cd ..** (Change to the parent directory); **cd ~**(change to Home directory); **cd /**(change to root directory); **touch** hello.txt (Make a new file named hello.txt); **mv** comp9001 hello.txt (Move hello.txt to comp9001); **rm** comp9001/hello.txt (Remove a file named hello.txt in comp9001 directory); **rmdir** comp9001 (Remove a directory name comp9001); **Ctrl + C** to stop infinite loop in program.

Compilation is the process of translating the entire source code into machine code **at once**, which is typically used for developing most software applications. Its main **advantages** include high performance since there is no translation overhead during execution, and the fact that recompilation is unnecessary for future uses unless changes are made. **However**, its downsides are that compiled programs can only run on the specific hardware they were compiled for, and their **error detection is relatively weaker** compared to interpreted languages. **Java**
Interpretation, on the other hand, translates and executes the source code **line-by-line** during runtime and is commonly used for generating web-based scripts. The primary **benefit** of this approach is that it makes **testing and debugging much easier** for developers to pinpoint problems, while also allowing for dynamic typing to support more programming techniques. **Nevertheless**, it suffers from **slower performance** due to the overhead of ongoing translation during execution, and interpreters cannot optimize code as effectively as compilers because they do not analyze the entire code beforehand. **Python**.

Symbol:
 \ : Escape Character
 print(): print("\n\n\n") will print 4 lines; print(123, end="") will only print 1 line; print("line1","line2", sep="\n")

Arithmetic Operators:
 + : Addition
 1 + "1" TypeError
 [1] + "1" TypeError
 [1,2] + 3 TypeError
 "1" + "2" = "12"
 [1,2] + [3] = [1,2,3]
 (1,2)+(3) TypeError
 1+True = 2
 10 + False = 10
 - : Subtraction
 "ab" - "a" TypeError
 [1,2] - [1] TypeError
 * : Multiplication
 [1,2]*3 = [1,2,1,2,1,2]
 [1]*-1 = []
 "ab"*3 = "ababab"
 "a"*-1 = ""
 / : Division, always produce float
 ** : Exponentiation (raising a number to a power)
 % : Modulus (finding the remainder left over after division)
 // : Floor division (dividing and rounding the result down to the nearest whole number)
 Order of preference: ** -> *, /, //, %, -> +, -
 Examples:
 10/4 = 2.5
 10//4 = 2
Logic Operators: (lower case heading letter)
 not
 and
 or
 Order of preference: not->and->or
Relational / Comparison Operators

==
0026
>
List1>list2 #比较 list1 中第一个 element 和 list2 中的第一个 element, 如果相同, 则比较第二个 element。

Syntax Rules:
Variable Name: must start with _ or a letter (no number); Only contain letters, numbers, underscore (no space, -)
PEP-8 Convention: first, variable names should avoid clashing with existing Python built-in objects or functions (though no option had this issue); second, they must be in all lowercase with words separated by underscores; Third, names should be descriptive of the value that it is representing.

Loop
Count-Controlled: 明确知道它会执行多少次
 Better use for;
 while x < 5
 for i in range(3)
Event-Controlled: 无法预知循环会执行多少次
 Better use while;
 while keep_running == True

Function:
def <func_name> (<parameters>; <Type hint>;):
 return
Parameters: Are the var inside the ()
Arguments: values that passed to the function
Type hint: Python does not enforce typing.
Return: 不写默认返回 None, return 是 not produce Any output of, print 会
Side Effect (Impure Function): Occurs when a function modify some state outside of its scope. Such as
 Changing global variable
 Modify mutable argument
 Print() and input()

Global Variables and Local Variables
def func():
 result = x + 10
 return result
这里 x 就是 global, result 就是 local
但是如果你想改变 global var 需要在 func 里面声明, 否则有 UnboundLocalError
def func():
 global x # 只能独立行声明
 x += 20

String (Immutable):
 Only + and * arithmetic operator works.

Slicing:
 # 可以看到是包含右边界 idx=6 的
 >>> s = "0123456"
 >>> s[0:6:2] '024'
 # 可以看到每个正数 index 都有其对应的负数 index.
 # 比如"0"的 idx 是 0 又是-7
 # 比如"6"的 idx 是 6 又是-1
 >>> s[-1:1:2]
 '024'
 >>> s[-6:-1]
 12345
 # 可以看到当 step 为负数时, string 是反过来读取的
 >>> s[:2]
 '6420'
 >>> s[0:0]
 "
 >>> s[6:1]
 ""
 >>> s[:n] # exclude the last n elements from s

Format String:
 {:2f}: 2 decimal places
 {:04}: 0 means pad with zeroes, width should be 4
 {:4}: pad with space, width should be 4
 {:>4}: pad with *, width should be 4. Such as ****1
 {:<4}: pad with *, width should be 4. Such as 1***
 {:=4}: pad with *, width should be 4. Such as *1**
f-string:
 >>> x = "Amy"
 >>> y = 26
 >>> print(f"Hello{x}, I am{y} years old")
 Hello Amy, I am 26 years old

>>> print(f"{y:04}")
0026
str.format():
 >>> x = "Amy"
 >>> y = 26
 >>> print("Hello {}, I am{ } years
 old".format(x, y))
 Hello Amy, I am 26 years old
 >>> print("{}{:04}".format(y)
 0026

Method: str.method()
swapcase(): Return a copy of the string with uppercase characters converted to lowercase and vice versa.
strip(): Defaultly, it will strip all the whitespaces on the left and right of the users input.
split(sep=None, maxsplit=-1): Return a list of the words in the string, using sep as the delimiter string.
If maxsplit is given, at most maxsplit + 1 splits are done.
 >>> '1,2,3'.split(',')
 ['1', '2,3']
lower(): Return a copy of the string with all the cased characters converted to lowercase.
replace(old, new[, count]): Return a copy of the string with all occurrences of substring old replaced by new: If the optional argument count is given, only the first count occurrences are replaced.
find(sub[, start [, end]]): Return the lowest index in the string where substring sub is found within the slice s[start:end]. -1 if not found.
capitalize(): Return a copy of the string with its first character capitalized and the rest lowercased.
count(value[start,end]): returns the number of times a specified value appears in the string.
isalpha(): return True if all characters in the string are alphabetic and there is at least one character, False otherwise.
isnumeric(): 如果字符串中只包含数字字符, 则返回 True, 否则返回 False

Conversion:
 str("1,2") = "1,2"
 str("1") = "1"
 str("1") = "1"
 str({1:1})="1:1"

Integer (Immutable):
Conversion:
 int(2.5) = 2
 int("03")=3
 int("1.2") ValueError
 10/2 = 5.0 # Always produce float

Float (Immutable):
Conversion:
 float("2.5") = 2.5
 float("2") = 2.0

List (Mutable): Ordered Items have a defined order that will not change. If you add new items, they are placed at the end. **Mutable**: You can change, add, and remove items after the list has been created. **Allow Duplicates**: Lists can have multiple items with the exact same value. **Heterogeneous**: A single list can contain different data types simultaneously (e.g., integers, strings, and booleans together).

Slicing: same as string slicing; it makes a copy of original list.
Jagged or Irregular list: Some 2D list have rows of different lengths.
[[1],[1,2]]

Method: list.method
append(): Add an item to the end of the list.
extend(): adds all elements in one list to another list.
list.sort(*, key=None, reverse=False): Sort the items of the list in place. F means Asc, T means Desc
remove(element): removes the first element equal to the argument. 找不到为 ValueError
insert(index, element):向本来 list 中 idx 的位置添加 element.
 >>> x = [1,2,3] >>> x.insert(1,1.5) >>> x [1, 1.5, 2, 3]
reverse(): Reverse the elements of the list in place.
pop(index): 会返回值; if did not specify the digit (count from beginning), it will remove and returns the last element.
count(x): Return the number of times x appears in the list.

Conversion:
 list("abc") = ["a","b","c"]
 [None, 1]

Array(Mutable): In python list can hold different data types, but arrays are used to store the same data type. 不是 built-in, 需要 import.
Tuple (Immutable): It similar to list, except it is immutable. Try to modify a tuple will lead to TypeError (tup[0]=1 TypeError). So no append, remove, pop method. (1,2,3)
 >>> x = 1,
 >>> type(x)
tuple
Dictionary(Immutable): {key1:value1,key2:value2}
keys must be unique and immutable; values can be any and be duplicated.
for key in dict 和 for key,value in dict 都可以
 >>> 1 in {2:1}
False
Methods:

clear(): clear all items from the dictionary.
get(key[, default]): Return the value for key if key is in the dictionary, else default. If default is not given, it defaults to None, so that this method never raises a KeyError.
keys(): Return a new view of the dictionary's keys. 返回的是 view type, 也就是说我们要通过 list()函数把它转换成 list
 >>> my_dict = {"a": 1, "b": 2}
 >>> keys_view = my_dict.keys() # 这是一个视图对象
 >>> my_dict["c"] = 3 # 修改原字典
 >>> print(keys_view) # 输出: dict_keys(['a', 'b', 'c']) -> 列表
视图自动感知到了 "c"!
values(): Return a new view of the dictionary's values. An equality comparison between one dict.values() view and another will always return False. This also applies when comparing dict.values() to itself.
items(): Return a new view of the dictionary's keys.
pop(key,default): If key is in the dictionary, remove it and return its value (not key), else return default.

class:
Instance Attribute:
 不包括 self, 但是只要通过 self.var = xxx 的, 都属于。
Class Variable:
在 class 里面的 global var. 更改一个实例的 class var 不会导致其它实例的 class var 进行改变, 而是针对本身新建一个 Instance Attribute. 你要更改必须使用 class 本身才行。
class Car:
 wheels = 4
 car1 = Car()
 car2 = Car()
 Car.wheels = 6
 print(car1.wheels, car2.wheels) # both 6
 car1.wheels = 8
 print(car1.wheels, car2.wheels) # 8 (ins at) 6 (class var)

Instance Method: using self. **"class.instance.method()"**
Static Method/Function: do not use self. **"func(class_instance)"**
Built-In Variable:
 __name__:
 In current module, __name__ == __main__
 In other module, __name__ == module name
 >>> import math
 >>> print(__name__)
 __main__
 >>> print(math.__name__)
 math
 __eq__: Default, it compares the address of two class instances. If you define it, it will override the default one.
 x = y means x.__eq__(x)
 __str__: Default, it returns the address of class instance. If you define it, it will override the default one.

Error:
Syntax Error: happens when the Python interpreter cannot compile your code. Such as print("a"), IndentationError, TableError.
Exception: Happens during execution.
 NameError: Try to use a Var that does not exist
 IndexError
 ValueError: The type is correct, but value is not
 TypeError: The type is incorrect
 RecursionError
 Raise: 如果你没有用 try...except...捕获, 则 raise 会导致程序终止. "raise ValueError("Invalid Input")"
Read/Write/Append File
 'r' for read, 'w' for write, 'a' for append.
 Read not exit file will lead error.

Write not exit file 会在指定路径下自动创建一个新文件, 然后把内容写进去。
 会覆盖原本的文件。
 需要手动在末尾添加换行符, 不会自动添加。
 f.write("abc\n").
Append not exit file 和 Write 模式一样不会报错。
 不会覆盖原本的文件。
 需要手动在末尾添加换行符, 不会自动添加。
 f.write("abc\n").
Open Method:
 #方法一, 需要手动关闭
 f = open("foo.txt", "r")
 f.close()
 #方法二, 不需要手动, 并且即使报错, return 同样会关闭 file.
 with open("foo.txt") as f:
 ...

Function&Method: f = open("foo.txt", "r")
print(f): 只会打印 f 存储的内存地址
f.read(): 将文件的所有内容一次性读取并作为一个字符串返回 读取文件 - Python 教程. 如果文件非常大, 此方法可能会占用过多内存 读取文件. 支持传入参数 f.read(size) 来限制读取的字符串
f.readline():每次只读取文件的一行内容, 并为字符串返回 (包含末尾的 \n 换行符)。从上一次读完的位置继续往下读的。
f.readlines():读取文件的所有行, 并返回一个列表, 列表中的每一个元素对应文件中的一行 (包含末尾的 \n 换行符)。更推荐下面的写法: "for line in f:", 这里 line 是 str 类型。
f.write(s): 一般通过 mode 确定是写入末尾还是覆盖. s 需要手动加上 \n
f.writelines(list_of_s): 接收一个包含多个字符串的序列 (如列表), 并将它们依次写入文件中。

Test:
一个金字塔, 从上到下分别是 End To End Test, Integration Test, Unit Test. 上面的写起来最慢, 最复杂, 数量最少。
End To End: verify entire program or system works as a whole, from start to finish.
Integration: verify how different components work together. Might involve multiple classes.
Unit: Verify the smallest parts of your code, usually function or method. It is the **majority of test**
Black Box Testing focuses strictly on what the program should do, treating the software as an opaque box where the tester has no access to the underlying code. This blind testing approach is exceptionally effective at catching mismatches between the software's behavior and its functional specifications.
White Box Testing shifts the focus to how the program operates internally. It involves writing test cases with full visibility into the source code, allowing testers to target specific control flows, execution logics, or lines of code suspected to contain hidden bugs.

Doc string Test:
def square(x):
 """
 >>> square(3)
 9
 """
 return x * x
import doctest
result = doctest.testmod()
print(f"失败了 {result.failed} 个测试")
Assertion:
assert <condition>, "Optional Error msg"
返回 False 表明输出不符合预计
Test Driver:
Import the functions you want to test, define a function for each test case, use assert, call those functions under an if __name__ == "__main__" from program import some function
def test_adds_positive_numbers():
 assert ..., "fail"
 print("pass")
 # each function should focus on 1 thing

Recursive
Without a base case will not run out memory but raise RecursionError.
def fib(n):

```

if n == 1:
    return 0
if n == 2:
    return 1
else:
    return fib(n-2) + fib(n-1)

```

Flowchart:

